



TAPIA 2026

SEPTEMBER 16-18, 2026 | ATLANTA, GEORGIA



Association for
Computing Machinery

Hackathon Challenge Problems

Welcome to the 2026 TAPIA AI Hackathon! You're here to build, break, and create in ways you might not have thought possible. Over the next 4 hours, you'll have the chance to take on a challenge, push your creativity, and work together in a team to bring wild ideas to life.

Remember: this isn't just about who wins. It's about exploring, learning, and surprising yourselves with what you can do. The judges are here to cheer you on and celebrate the risks you take, not just the final product."

✓ What Success Looks Like Today

By the end of the hackathon, you'll know you crushed it if your team:

- **Pushed the challenge further** – You didn't just stop at the basics, you added something new or unexpected.
- **Brought creativity to the table** – Your idea, your design, or your approach makes people go, "Whoa, that's clever!"
- **Showed real technical chops** – You went beyond concepts and built something that works, even if scrappy.
- **Worked together like a pro team** – Everyone contributed, everyone had a voice, and your final presentation showed that teamwork.
- **Checked the agent's work** – You can point to something the agent got wrong, and explain how a (human) person on your team caught it.

"So dive in, experiment boldly, and don't be afraid to chase the crazy ideas — because those are often the best ones. Let's hack!" 🚀

Pick one challenge for your team. Use open-source data, including the sources suggested with each challenge, if you need sample data. Every challenge below can be built with Hermes Agent and the LLM providers from your setup guide.

[1] Open your LLMs for Social Impact

Create an Agentic AI solution designed for social impact. Choose a track – Education, Environmental Justice, or Healthcare Accessibility – and build an autonomous system that perceives its environment, makes decisions, and uses external tools or APIs to take real-world action on behalf of marginalized or underrepresented groups. Go beyond a chatbot that retrieves information: your agent should take a high-level goal, break it into steps, call external APIs, handle errors, and execute a sequence of actions that removes administrative friction, advocates for a user, or automates a complex bureaucratic process.

Datasets & resources: data.gov and your city's open-data portal; [USA.gov](https://www.usa.gov) public benefit program pages; OpenStreetMap (osm.org) for location and service data.

Show the judges: the goal you gave the agent, the steps it chose, at least one real action it took, and how a human teammate confirmed it did the right thing.

[2] Generative AI Neurodiversity-Adaptive Study Workspace

Develop an AI-driven tool that dynamically restructures academic textbooks or technical documentation into visual flowcharts, checklists, or gamified summaries tailored for ADHD or dyslexic learning styles. Extend it with agentic capability where the agent should decide *which* restructuring fits the material and the learner, produce it, check its own output against the source, and adapt when the student says it isn't working.

Datasets & resources:

- Academic textbook: 3 complex paragraphs from an OpenStax Biology chapter (e.g., Cellular Respiration), <https://openstax.org/>
- Technical documentation: AWS IAM *Policy evaluation logic* ([here](#)) or the React *useEffect* reference ([here](#))
- Dense STEM concept: OpenStax Calculus Vol. 1, §3.6 The Chain Rule ([here](#)) or University Physics Vol. 2, §6.3 Applying Gauss's Law ([here](#))

Show the judges: the same passage rendered two different ways for two different learners, and a human teammate's side-by-side check against the source showing nothing was lost or invented.

[3] Interactive Detective Roleplay Game

Develop an AI-powered text-based mystery game in which AI personas act as independent characters, each with goals, secrets, speech quirks, and background lore, that interrogate and interact with each other and with a human player to uncover hidden clues and solve the case. The agentic part: characters should hold their own state and secrets across turns (memory management), react to what other characters reveal, and never leak information their persona wouldn't know.

Datasets & resources:

- PersonaHub (Hugging Face): 200,000 persona profiles, including 10,000 game-NPC profiles: <https://huggingface.co/datasets/proj-persona/PersonaHub>
- SimsConv: 68 characters, 1,360 scenes, 13,000+ roleplay dialogues: <https://github.com/Bernard-Yang/SimsChat> (paper: <https://arxiv.org/abs/2406.17962>)
- Synthetic-Persona-Chat: <https://github.com/google-research-datasets/Synthetic-Persona-Chat>

Show the judges: a live playthrough where a character keeps a secret under pressure, and a human teammate who knows the full solution, confirming that no character leaked what it shouldn't have known.

[4] Lost in Translation: Idiom & Slang Explainer

When consuming media across languages or cultures, the hard part isn't the words. The idioms, regional slang, cultural metaphors, and generation-specific jargon (Gen Z slang, technical jargon, regional dialects) can trip up any non-native speaker. Build a tool that translates audio, video, or text between languages or subcultures and detects and explains these expressions in an easy-to-digest format. Make it agentic: the agent should recognize when a literal translation is misleading, go look up the expression, and decide how to explain it for this audience.

Datasets & resources: Wiktextextract / kaikki.org (structured Wiktionary); wiktionaryparser (pip); Open Parallel Corpora (e.g., giga-fren v2); any short public-domain audio or video clip.

Show the judges: three expressions a plain translator would get wrong, and what your agent did instead. Have a human teammate who speaks the language or knows the slang verify the results.

[5] Your Movie Advising Agent

Create an agentic movie advisor system that goes beyond standard filtering. Given a user's preferences, the agent searches open movie databases, analyzes reviews and critiques, cross-references ratings from multiple sources, and produces a prioritized watch list that explains the reasoning behind each pick. Let users track movies they've watched and movies their friends and family have watched. When a new movie is added to a list that the user follows, an agent automatically compares it with the user's history, finds similar movies by rating and genre, and recommends whether to watch it and others like it.

Datasets & resources:

- TMDB 5000 Movie Dataset ([here](#))
- The Movies Dataset: ([here](#))

Show the judges: a recommendation with its reasoning, and what changed when a friend added a movie, plus one recommendation a human teammate disagreed with and why.

[6] CivicSense: Real-time Intelligence for Real Life

When a crisis hits — a storm, a wildfire, a transit meltdown, a hazardous-air day — the information exists, but it's scattered, raw, unverified, and overwhelming. Build an AI agent that turns chaotic live public data into clear, verified, personalized guidance for the person who needs it right now: a citizen ("I'm at 5th & Main. Is it safe to walk home?"), a caregiver ("Is my kid's school route affected?"), or a wildland firefighter ("My engine is staging on Forest Road 12 — where's the fire edge now, what's the wind doing in the next three hours, and is my escape route still clear?"). This is not a chatbot. The agent must *plan* which sources matter, *fetch* from multiple live feeds, *verify* across sources so it never acts on a single unconfirmed report, *reason* about severity for this persona, *guide* with plain-language steps, and *loop* by rechecking and updating as things change.

Datasets & resources: (all free, live, no sensitive data)

- *Wildfire*: NIFC WFIGS: official named incident locations ([here](#)) and current perimeters ([here](#)) (ArcGIS REST/GeoJSON, no key needed); NASA FIRMS: satellite hotspots ([here](#)), minutes old (free map key); NASA's Python tutorial ([here](#)). NOAA HMS: analyst-curated detections ([here](#)), refreshed every 10 min (no key needed); InciWeb: official incident updates ([here](#)); CAL FIRE incidents ([here](#)) (no key needed)
- *Weather & air*: NWS alerts including Red Flag Warnings ([here](#)); Open-Meteo: wind forecast at a point ([here](#)) (no key needed); AirNow ([here](#)) and the Fire & Smoke Map ([here](#)); OpenAQ: smoke and air quality ([here](#)) (free key)
- *Other hazards*: USGS real-time earthquakes ([here](#)); GTFS-Realtime transit alerts ([here](#)) plus a local agency feed for Atlanta: MARTA developer resources ([here](#)); Traffic for Georgia 511 ([here](#)); Civic issues: Open 311 ([here](#)) or your city's open-data portal.

Show the judges: throw a live or simulated crisis at it and watch the agent pull real feeds, cut the noise, tell a scared human what to do, and update when you change the scenario. Point to one report it marked *unconfirmed* and explain how a human would settle it.

NOTE: These feeds are already live, so you just need your agent to poll them. Start with one persona and two feeds. A satellite hotspot with no matching NIFC incident is exactly the kind of “unconfirmed” report your agent should flag, not act on. This is a training exercise, which is never a substitute for incident command.

[7] Translator: Plain Language or Another Language, Without Losing Facts

The documents that decide who gets aid, housing, or medication are written for lawyers, and mostly in English. Take a dense public document — financial-aid instructions, a medication guide, a lease, a university policy — and build an agent that transforms it *while a second agent checks that no fact, number, deadline, or condition was dropped or changed*. Choose a track: **Plain language** (rewrite at a 6th–8th-grade reading level), or **Another language** (Spanish, Vietnamese, Haitian Creole, or whatever a human teammate speaks, and add plain-language in this target). If your team has a bilingual member, pick the second track: they're the ground truth no agent can replace.

Datasets & resources:

- Financial aid: *Filling Out the FAFSA Form* ([here](#)) and the official Spanish version ([here](#)), a free reference translation to compare against
- Medication: FDA Medication Guides index ([here](#)), pick any common drug; many have Spanish versions on the label page
- University policy (Atlanta): Georgia Tech Academic Honor Code ([here](#)) or the longer Student Code of Conduct ([here](#))
- Readability: `textstat` — <https://pypi.org/project/textstat/> (`pip install textstat`)

Show the judges: the original beside the transformed version, the list of facts your verifier checked, and one thing the agents got wrong that a human caught.

[8] Course Study Companion: From Syllabus to Spaced-Repetition Quiz

Build an agent that helps a student learn faster and better during an academic course. Give it a syllabus and a chapter (or a research paper), and have it produce a concept map, a critical-review sheet, and a quiz. Then, it remembers what the student got wrong and re-quizzes those items in a later session. Make it agentic: multi-phase, with the human student in the loop, and with memory that survives across sessions. Optional inclusion angle: a learning-style mode (dyslexia-friendly formatting, ADHD-paced chunks, screen-reader-friendly output).

Datasets & resources: any OpenStax chapter (<https://openstax.org/>); "Attention Is All You Need" or any arXiv paper; the `paper-study` skill from the AI Agentic Tutorial shared folder as a starting point. Read about a real solution developed by researchers at Stanford University ([paper](#)) ([online interface](#)).

Show the judges: quit the agent, relaunch it, and have it re-quiz only what the student missed, with the quiz answers checked by a human teammate against the source, not by the agent.

[9] Data Janitor: Audit a Messy Dataset

Hand the agent a CSV riddled with problems — duplicates, unit mismatches, impossible dates, mixed encodings, silent nulls — and have it produce a data-quality report, propose fixes, apply them to a clean copy, and document what it changed and why. The agent has to inspect, hypothesize, test, and iterate: profile, check, fix, re-profile. No fix without a stated reason.

Datasets & resources: Two ways to get a messy file.

- *Plant your own* (recommended because you build the answer key): take a clean slice of a real dataset and have two human teammates inject defects while the rest of the team doesn't look. Chicago Data Portal ([here](#)); NYC Restaurant Inspections ([here](#)); Open Food Facts ([here](#)) (only use a sample slice because the full export is huge).
- *Pre-dirtied practice sets* (free Kaggle account): Cafe Sales ([here](#)) (10k rows, classic planted problems); FIFA 21 Messy Raw ([here](#)) (19k rows × 77 columns of real scrape mess; this one is harder with no answer key).

Show the judges: the defects your agent found, the ones it missed, and any it invented that weren't really there. The human teammates who planted the defects are your answer key. If you used a pre-dirtied set, a human still has to rule on every one.

[10] Accessibility Fixer: Find and Repair WCAG Issues in a Real Site

Point an agent at a small open-source web project to run an accessibility scanner. The agent should prioritize the findings, fix what it can (e.g., alt text, contrast, form labels, heading order, keyboard focus), re-scan to confirm, and write the pull-request description a maintainer would actually accept. Scan → fix → re-scan → iterate is the loop, then “did the fix work?” is checkable by machine. Alt text must be descriptive, not “image.”

Datasets & resources:

- *Scanners* (need [Node.js](#), see the workshop's setup guide's bonus page): pa11y ([here](#)) (`npm install -g pa11y`; downloads a headless Chromium) or axe-core CLI ([here](#)) (`npm install -g @axe-core/cli`). Both scan a URL, not a file. Serve your site locally first with `python -m http.server 8000`.
- *Practice targets with known violations*: A11yGoat ([here](#)); bada11y ([here](#)); the [example/](#) page [here](#) (20+ documented violations, the README is your answer key; don't peek until you've scanned).
- *Realistic targets (no answer key)*: any small MIT-licensed site template, e.g., from ([here](#)).
- *Reference*: WCAG 2.2 quick reference: <https://www.w3.org/WAI/WCAG22/quickref/>

Show the judges: before-and-after scanner output, and a human check that the fixes didn't break the layout or add alt text that's confidently wrong.

[11] Fake News Claim Checker: Fact-Check an AI Hype Article

Feed an agent an essay you find online about AI, the bigger or more controversial the claims, the better. It extracts every factual claim, classifies each as *supported in the text*, *verifiable externally*, or *opinion*. Then, the agent goes and checks the verifiable ones against outside sources and produces an annotated version with confidence badges. Extract → classify → search → compare → annotate. This is the most honest demo of an agent's limits: it *will* get some calls wrong, and your human-in-the-loop job is to show you noticed.

Datasets & resources:

- [Something Big Is Happening](#), Matt Shumer (Feb 9, 2026): a widely shared piece arguing that AI is about to change everything.
- Find your own essays as part of the challenge. Substack and Medium, company blogs, and newspaper op-eds are full of them. Bonus points for pairing it with a rebuttal so your agent has to weigh both sides, or for having the agent go find recent AI hype articles on its own, and then explaining to the judges why it chose the ones it did. Tools: You can use Hermes's web-fetch tool to pull essays and check sources, plus a scratch notes file where the agent logs every source it consulted.

Show the judges: the annotated essay, multiple verdicts you checked by hand, including at least one claim the agent got wrong and how you caught it. A team that says "it got everything right" hasn't looked hard enough.